

Modül 01: Spring AI Temelleri, Mimari ve Model Sağlayıcıları

Kurumsal Java'da Modern Üretken Yapay Zeka (2026 Baskısı)

Eğitmen: Yapay Zeka Mühendisliği Eğitim Ekibi

Teknoloji Yığını: Java 25/27 (LTS), Spring Boot 3.4/4.x, Spring AI 1.0+, Ollama

1. Neden Java ile Üretken Yapay Zeka?

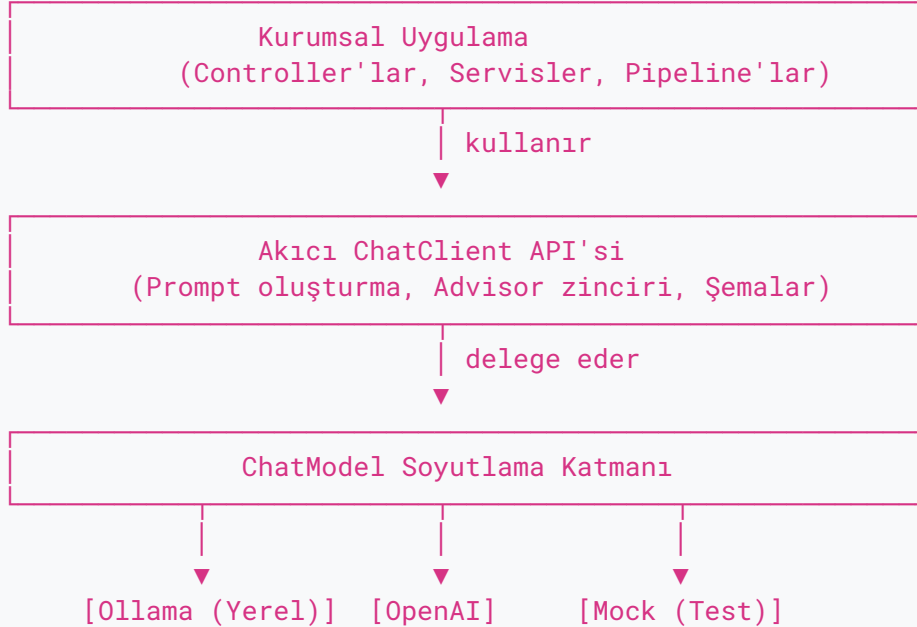
Kurumsal dünyadaki kritik iş yüklerinin büyük çoğunluğu Java üzerinde çalışır. İlk dönemlerde GenAI araçları daha çok Python ekosisteminde yoğunlaşmış olsa da, kurumsal üretim ortamları şunları talep eder:

- **Tip Güvenliği (Type Safety):** Derleme zamanında şema ve veri kontrolü.
- **Yüksek Eşzamanlılık (High Concurrency):** GIL (Global Interpreter Lock) engeline takılmadan on binlerce eşzamanlı oturum.
- **Gözlemlenebilirlik (Observability):** Micrometer, OpenTelemetry ve Jaeger ile uçtan uca izleme.
- **Ekosistem Entegrasyonu:** Güvenlik duvarları, kurumsal işlemler ve mikroservis mimarileri.

Spring AI, bu gücü modern Spring pratikleriyle birleştirerek Java geliştiricilerine sunar.

2. Spring AI Temel Mimarisi

Spring AI, iş mantığınızı alttaki model sağlayıcılarından tamamen soyutlar:



3. ChatModel ile ChatClient Arasındaki Fark

Evrim ve Mimari Ayrım:

- **ChatModel** (Alt Seviye - Low Level):
 - Sağlayıcı API'leri ile doğrudan 1'e 1 eşleşen arayüz.
 - Parametre olarak ham **Prompt** ve **Message** listeleri alır.
 - Çıktı olarak **ChatResponse** döner.
- **ChatClient** (Üst Seviye - Fluent & Modern):
 - Spring AI 1.x ile sunulan akıcı ve bildirimsel (declarative) istemci.
 - Varsayılan sistem talimatları, loglama, bellek (chat memory) ve güvenlik filtreleri (advisors).
 - Tip güvenli nesne dönüştürme desteği (**entity()**).

4. Akıcı ChatClient Örneği

```
@Service
public class AsistanServisi {

    private final ChatClient chatClient;

    public AsistanServisi(ChatClient.Builder chatClientBuilder) {
        this.chatClient = chatClientBuilder
            .defaultSystem("Sen kıdemli bir Java ve Spring AI mimarisin.")
            .defaultAdvisors(new SimpleLoggerAdvisor())
            .build();
    }

    public String soruSor(String soru) {
        return chatClient.prompt()
            .user(soru)
            .call()
            .content();
    }
}
```

5. Model Taşınabilirliği ve Dinamik Yönlendirme

Uygulamanızı asla tek bir yapay zeka sağlayıcısına bağımlı (vendor lock-in) kılmayın. **Model Routing Deseni:**

```
@Service
public class ModelRoutingService {
    private final Map<String, ChatModel> modeller;

    public ModelRoutingService(ApplicationContext context) {
        this.modeller = context.getBeansOfType(ChatModel.class);
    }

    public ChatClient getClient(String saglayici) {
        ChatModel secilen = modeller.getOrDefault(saglayici, varsayilanModel);
        return ChatClient.builder(secilen).build();
    }
}
```

Tek satır kod değiştirmeden yerel Ollama (`llama3.2`), OpenAI (`gpt-4o`) veya offline Mock arasında geçiş yapabilirsiniz!

6. SSE ile Reaktif Token Akışı (Streaming)

Kullanıcı deneyimi açısından yanıtın tamamını beklemek yerine kelimelerin geldikçe ekrana yazdırılması (streaming) esastır:

```
@GetMapping(value = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<ServerSentEvent<String>> stream(@RequestParam String prompt) {
    return chatClient.prompt()
        .user(prompt)
        .stream()
        .content()
        .map(token -> ServerSentEvent.<String>builder()
            .data(token)
            .build());
}
```

Spring WebFlux ve Spring AI, arka planda işletim sistemi thread'lerini bloklamadan token akışı sağlar.

7. Java 25/27 Sanal İş Parçacıkları (Virtual Threads)

Modern Java, hafif sıklet Sanal İş Parçacıkları (**Project Loom**) sunar. Birden fazla modeli eşzamanlı kıyaslarken:

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    List<CompletableFuture<ProviderResult>> futures = modeller.stream()
        .map(model -> CompletableFuture.supplyAsync(
            () -> modeliCagir(model, prompt), executor
        ))
        .toList();

    CompletableFuture.allOf(futures.toArray(new CompletableFuture[0])).join();
}
```

- Minimum bellek tüketimiyle milyonlarca sanal iş parçacığı açılabilir.
- LLM yanıtı beklenirken geçen uzun ağ I/O süreleri işletim sistemi iş parçacıklarını kilitlemez.

8. Ödev 01 Yönergesi ve Uygulama

"Çoklu Model Sağlık ve Karşılaştırma Servisi"

1. `module-01-foundations-and-models/homework/starter` dizinine gidin.
2. `MultiModelComparisonService.java` dosyasını inceleyin.
3. `compareAcrossModels` metodunu Virtual Threads ile uygulayın.
4. Sağlayıcı zaman aşımının (timeout) tüm servisi çökertmediğinden emin olun.
5. `mvn test` komutunu çalıştırarak tüm testlerin geçtiğini doğrulayın.